The Midterm, Step by Step

Midterm explanation and preliminary research

TODAY

0–45 min	The midterm, part by part, and your questions
45–95 min	Research lab in Colab
95–100 min	Save, share, and submit your Colab link

What the midterm asks you to show

Run a small PostgreSQL database safely, prove what happened, and explain it in your own words.

1. **Check the data** and explain three integrity controls
2. **Roll back** one change and **commit** another
3. **Explain one blocking incident** from the Week 5 lab
4. **Test one improvement**; the recommended one is an index
5. **Plan a recovery** after the Week 8 lesson

Individual · 30% of the course grade · midterm.sql + README.md in one ZIP · Deadline in Brightspace

What the starter gives you and what you add

Part	Already supplied	What you add
Setup	Three tables, all seed data, the count query	Run SECTION 1 once and record the counts
1 Data checks	Constraints you can explain	**The requester join**, then three controls explained in the README
2 Transactions	The complete rollback example, 2A	**The commit example, 2B**, then six observed values
3 Blocking	The Week 5 notebook and its supplied trace	A written explanation in the README
4 Improvement	The whole index experiment, 4A to 4C	Your IDs, plan lines, index size, and explanation
5 Recovery	The Week 8 lesson and its commands	A short recovery plan in the README

You write **two small pieces of SQL**: the requester join and the commit example. Most of your grade comes from **recording real results and explaining them**.

Two files: what goes where

midterm.sql · what you ran

- The supplied setup and count query
- Your requester join
- Rollback 2A and your commit 2B
- The index experiment, 4A to 4C
- Optional expected-error tests, commented out

README.md · what happened and why

- Run order and environment
- Counts, join result, and three controls
- Six transaction values and an explanation
- The blocking record and explanation
- Index results, one cost, and a recommendation
- The recovery plan, after Week 8
- References and assistance

Submit **one ZIP** with both files. No passwords, connection strings, or API keys anywhere.

Run the starter safely and see every result

1. **Practice database only.** SECTION 1 drops and rebuilds the midterm_metro schema.
2. **Run SECTION 1 once.** Then highlight one section and run only that section.
3. **Do not rerun SECTION 1 midway.** It resets ticket 1004 to low and removes the index.
4. **Keep each transaction together.** Run it from its first SELECT to its last as one block.

Need a clean start? Rerun SECTION 1, then repeat Parts 2 and 4 in order.

Seeing every result

The Supabase SQL Editor may show only the **last result** of a run. That hides the value inside a transaction.

For Part 2, use the `run_sql` Colab helper in the midterm guide, or a `psql` session. Both print every result in order.

Part 1: Load and check the data

STEP 1 · RUN SECTION 1

table_name	row_count
users	8
tickets	12
ticket_events	21

If your counts differ, rerun SECTION 1 before going further.

STEP 2 · WRITE THE REQUESTER JOIN

```
SELECT t.ticket_id, u.display_name AS requester
FROM midterm_metro.tickets AS t
JOIN midterm_metro.users AS u ON ______ = ______
ORDER BY t.ticket_id;
```

Each ticket's `requester_id` matches a `user_id` in users.

Expected: **12 rows**, one per ticket

First: 1001 · Requester 101 Last: 1012 · Requester 104

README **Database checks**: record the three counts and what your join returned.

Part 1: Explain three integrity controls

Choose three **non-key** controls from the starter. PRIMARY KEY and REFERENCES do not count.

Control	Where it is in the starter	Your explanation answers
NOT NULL	Many columns, for example `tickets.subject`	Which missing value does it reject?
UNIQUE	`users.email`	Which duplicate does it reject?
CHECK	tickets: `closed_at IS NULL OR closed_at >= opened_at`	Which impossible pair of dates does it reject?

ONE COMPLETE ANSWER

NOT NULL on tickets.subject rejects a ticket whose subject is missing (NULL). Without it, a ticket could enter the queue with no recorded problem.

Control · column · the bad data it stops. Write your three in your own words.

Part 2: Roll back, then commit

```
-- 2A is supplied. Run all six together.
SELECT ticket_id, priority ... ;  -- before
BEGIN;
UPDATE ... SET priority = 'high' ... ;
SELECT ticket_id, priority ... ;  -- inside
ROLLBACK;
SELECT ticket_id, priority ... ;  -- after
```

2B is yours to write

1. Copy the six statements.
2. Change `'high'` to `'medium'`.
3. Change `ROLLBACK` to `COMMIT`.
4. Run it as one block.

Test	Before	Inside the transaction	After rollback or commit
Rollback	low	high	low
Commit	low	medium	medium

> Run 2A before 2B. Leave **medium** in place: your recovery checks expect it.

Part 3: Explain one blocking incident

EVIDENCE SOURCE · CHOOSE ONE AND LABEL IT

my live run | **my saved Week 5 run** | **supplied trace analysis**

Record six things

1. The diagnostic SQL, the `pg_stat_activity` query
2. The blocked session's PID
3. The blocking session's PID
4. The row involved
5. The action that released the block
6. The final row values

How to read the diagnostic row

The blocked session is waiting: its `wait_event_type` is `Lock`.

Its `blocking_pids` lists the session it waits for. That session is the blocker.

Notebook example: `pg_blocking_pids(7332)` returns `{7310}`, so 7332 waits for 7310.

Different starting values: the Week 5 `lock_lab` ticket 1004 starts at medium and open, not low and new. Use the notebook's values.

Part 4: Run the index experiment

4A · BEFORE

Run the SELECT alone and save the IDs. Run the EXPLAIN alone and save the plan.

4B · CREATE

Create the index once. If it already exists, drop it with the starter's DROP INDEX line and repeat 4A to 4C.

4C · AFTER

Run the same SELECT and EXPLAIN again, then the size query.

Evidence	Before the index	After the index
Ordered IDs	1011, 1007, 1001	1011, 1007, 1001
Scan line in the plan	Seq Scan on tickets	Seq Scan on tickets
index_bytes	not applicable	16384, which is 16 kB

MEASURED ON POSTGRESQL 16

Your timings and cost numbers will differ. Cost can drop after CREATE INDEX even when the plan does not change, so compare the **scan type**.

Part 4: An unchanged plan is a correct result

Why a Seq Scan wins here

All 12 tickets fit on **one 8 kB page**. Reading that page is cheaper than reading the index and then the table.

An honest explanation of an unchanged plan can earn full credit.

Do not add fake rows or change planner settings to force a result.

In README **One improvement**, cover:

1. **The problem:** finding the newest open tickets as the table grows
2. **Did the plan change?** Quote the scan line from each plan
3. **Did the result stay the same?** Compare the ordered IDs
4. **One cost:** storage, plus extra work on inserts and on updates to indexed columns
5. **Your recommendation:** keep or drop the index, and why

Alternative routes, one improvement only: least-privilege grant · RLS policy test · safe schema change

Part 5: Write a recovery plan after Week 8

Source	practice database, schema `midterm_metro`
Backup	a logical `pg_dump`, saved as `midterm_metro.dump`
Destination	a separate, empty database, never the source
Restore	`pg_restore` into that destination
Checks	at least three, each with an expected result
Limits	roles, grants, other schemas, Supabase settings

```
pg_dump --format=custom \
  --schema=midterm_metro --no-privileges \
  ... --file=midterm_metro.dump
```

Check	Expected result
Row counts	8 / 12 / 21
Requester join	12 rows
Ticket 1004	priority medium
Index	still present

> Status line: write **proposed plan** unless you ran it. Running the restore is **optional**.

Your README, section by section

README section	Part	What goes there
Run order	All	How to run midterm.sql from the top, in what order, and what the SECTION 1 reset deletes
Database checks	1	Counts, the join result, and three controls
Transactions	2	The six observed values and your explanation
Blocking	3	Evidence label, diagnostic SQL, both PIDs, the row, the release, and the final values
One improvement	4	Ordered IDs, plan lines, index size, the explanation, and one cost
Recovery plan	5	Status line, source, backup, destination, restore, checks, and limits
References and assistance	All	The starter, notebooks, documentation, and any assistance you used

At the top: your name, section, and environment. `SELECT version();` prints your PostgreSQL version. No credentials.

Observed results versus predictions

OBSERVED

“My rollback test showed low before, high inside the transaction, and low after ROLLBACK.”

PREDICTION OR PROPOSAL

“I predict that a restored copy will contain 12 tickets. I have not run this restore.”

- Copy values from **your own output**, not from the slides or the guide.
- Label anything you did not run as a **prediction** or **proposal**.
- Label supplied trace values as **supplied**, not as your run.
- If a step failed, **say what failed**. Never invent a result.

How the rubric reads your work

Rubric area	Points	Evidence that earns it
Reproducible schema and data integrity	25	Setup runs; the 8 / 12 / 21 counts and the 12-row join are recorded; three controls are explained
Transactions and blocking diagnosis	25	Six observed values are explained; the blocking record matches its evidence and label
Workload-based improvement	20	A reason, relevant before and after checks, and one tradeoff (index route: IDs, plan lines, index size, one cost)
Recovery readiness	15	A separate destination and at least three checks with expected results
Documentation and communication	15	Another person can follow both files; every prompt is replaced; no secrets

> A faster time or an Index Scan is **not required**. A small working project with clear explanations is enough.

Common mistakes to avoid

- × **Rerunning SECTION 1** partway through the project
- × **Splitting a transaction** across separate runs
- × **Recording only the last result** the editor showed
- × **Copying the expected values** instead of your own output
- × **Calling the supplied trace** a live run
- × **Adding fake rows** or changing planner settings
- × **Pasting a password** or connection string into a file
- × **Leaving template prompts** in square brackets
- × **Submitting the Colab notebook** instead of the two files

> Check your work against the guide's **Expected results at a glance** table, then report your own values.

Timeline and questions

Today · Week 7	The midterm explained, then research in Colab. Submit your Colab link.
Week 8 · first meeting	Backup and restore lab: the basis for Part 5.
Week 8 · second meeting	Midterm clinic. Bring one question and one working part.
Deadline	midterm.sql and README.md in one ZIP, by the date in Brightspace.

Suggested order: Parts 1 and 2 first, then 3 and 4, then 5 after the backup lab.

Questions now, then open the Colab notebook.

Today's research lab

Open in Colab from the Week 7 README · **File > Save a copy in Drive** · rename it **Lastname_Week07_Research**

1 Inspect

ABOUT 10 MIN

Run the cells. Fill in the observation table. Record the open list before switching to new.

2 Read

ABOUT 10 MIN

The two PostgreSQL pages linked in the notebook, only the paragraphs it names.

3 Write

ABOUT 25 MIN

180–250 words across three cells. Label your prediction.

4 Check

ABOUT 5 MIN

Match every claim to an output above it.

No Supabase login or password: the notebook runs Python on data already inside it. Python results describe the data. They do not show how PostgreSQL runs the query.

Step 1: Inspect the data

ABOUT 10 MINUTES · MINUTES 45 TO 55

1. Run the code cells from the top. Record the **three counts**.
2. Run the filter with STATUS = "open". Record the **ordered IDs**.
3. Change STATUS to "new", run it again, and record **that list** too.
4. Inspect ticket 1004 or another ID. Record its **status and requester**.

Your observation table

Observation	Your result
Users / tickets / events	from your output
Ordered open-ticket IDs	before changing STATUS
Ordered new-ticket IDs	after changing STATUS
One ticket's ID, status, and requester	the ticket you inspected

> A blank assignee means **unassigned**; every ticket still has a requester. MAX_ROWS = 20 is a **ceiling**, not a promise that twenty tickets match.

Step 2: Read two short sources

ABOUT 10 MINUTES · MINUTES 55 TO 65

PostgreSQL: Indexes Introduction

Read the opening example through the paragraph that begins “Once an index is created,” then the final paragraph about keeping an index synchronized with its table. Skip the operator classes.

PostgreSQL: Indexes and ORDER BY

Read the first three paragraphs, through the discussion of ORDER BY with LIMIT. The later mixed-direction examples are optional.

Look for three things

What an index might let PostgreSQL **avoid**

Why PostgreSQL might **choose another plan**

What **extra work** an index adds

Both links are in the notebook. Paraphrase the ideas you use and keep both links for your writing.

Step 3: Write 180–250 words

ABOUT 25 MINUTES · MINUTES 65 TO 90

Response cell	Words	What it includes
A The question and data	50–70	What the query at the top of the notebook must return; two observations from your notebook; why LIMIT 20 can return fewer than twenty rows
B What the sources suggest	80–110	How an index could help the filter or the ordering; one cost; one idea from each source, linked; why an index definition online does not prove this database will run faster
C Your planned test	50–70	One before-and-after PostgreSQL test: what stays the same, which results and plan evidence you compare, what would make you keep or reconsider the index; a labeled prediction; one remaining question

Use your own words and **label your prediction**. Double-click a text cell to edit it. This writing becomes the start of **Part 4** in your midterm README.

Step 4: Check your work

ABOUT 5 MINUTES · MINUTES 90 TO 95

- Every row of the observation table comes from **your own outputs**
- Cells A, B, and C replace the prompts: **180–250 words** in total
- Both **source links** are still there
- Every observation matches an **output above it**
- Your expected test result is labeled a **prediction**
- If a cell failed, **say so**. Never invent an observation.

What I look for

- Accurate observations
- A connection between the two sources and this dataset
- A test that compares the same query and results
- A clear line between what you observed and what you predict

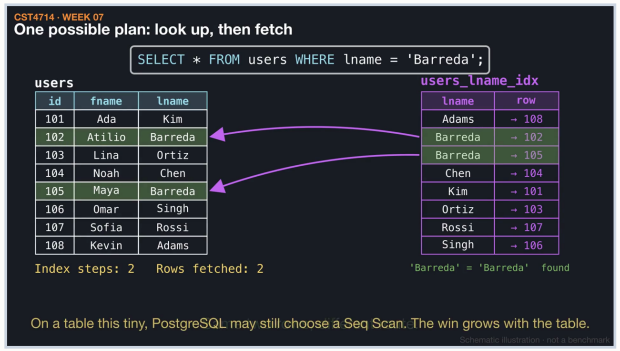
Save, share, and submit your link

1. Run every cell, then **File > Save**.
2. **Share**: give the instructor viewer access.
3. **Copy the link.**
4. Paste it into **today's classwork assignment** in Brightspace.

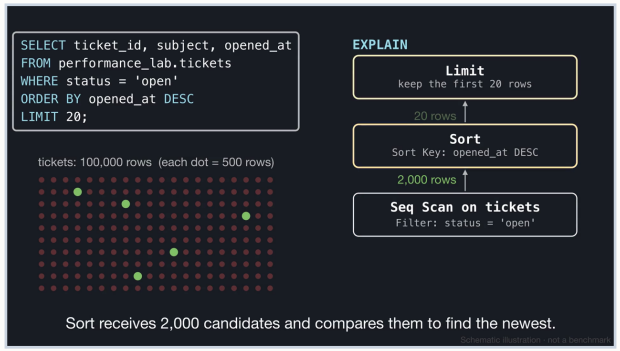
> Submit **your saved copy's link**, not the starter's GitHub or Colab link. No file upload or screenshots.

Next week: backup and restore, then the midterm clinic.

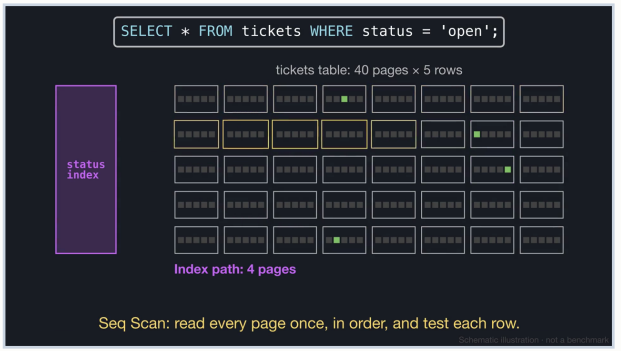
This week's videos



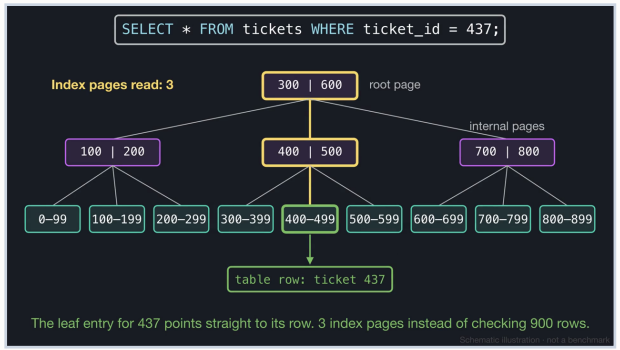
When does an index help? (1:44)



Reading a query plan (0:55)



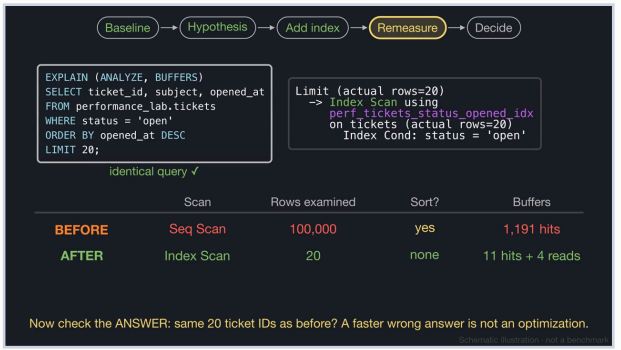
Selectivity: why a Seq Scan can beat an index (1:07)



How a B-tree index finds rows (0:58)



Composite indexes: column order matters (1:08)



Testing an index hypothesis (1:03)

Click a thumbnail to play on GitHub. Silent, with captions and .srt subtitles; diagrams are schematic.